

Multimodal Explanation System (Voice + Image): Product Vision and Requirements

We propose a **multimodal AI explanation system** that accepts an image or diagram plus a user query (via text or voice) and returns an integrated explanation: human-readable text accompanied by annotated visuals (highlighted regions, arrows, labels) and optional speech output. This system is inspired by “how a coach explains a move” – it analyzes the visual content (e.g. a diagram, flowchart or game board) and the user’s question, then “reasons” through a large language/vision model to generate a clear, step-by-step explanation, illustrated on the original image. This approach extends recent explainable-vision research: for example, TextVQA models now generate both a textual answer and a visual highlight of evidence ¹. Our system generalizes this idea beyond specific domains (like chess) to *any* field with visual content, integrating vision, language, and speech in one pipeline.

Key goals include: (1) **Domain-agnostic** explanations of diagrams, charts, and images (from science, engineering, education, games, etc.); (2) **Multimodal input** (support both typed text and spoken questions) and **multimodal output** (textual answer, synthesized speech, annotated visuals); (3) **Interactive visualization** (static diagrams annotated initially, with dynamic or interactive visuals as future extensions); (4) **Web-based, production-ready** deployment for use by students, educators or general users; (5) Leveraging **state-of-the-art AI** (vision-language models, speech engines, modern LLMs like GPT-4/GPT-4o) for high accuracy and usability. The system sits at the intersection of explainable AI, multimodal learning, and HCI, aiming to make complex content understandable through human-like reasoning.

Target Users and Use Cases

- **Learners and Educators:** Students or teachers can upload a diagram (biology circuit, math graph, chemistry structure, flowchart, etc.) and ask questions (“How does this motor circuit work?”). The system speaks and writes an explanation, highlighting key parts of the diagram. This emulates an interactive tutor or coach (as in the chess example) ¹.
- **Casual/General Users:** Anyone curious about an illustration can ask questions by voice or text (e.g. “What is happening here?” for a history map, or “Which strategy is best in this game state?” for any board game).
- **Accessibility:** By supporting voice I/O and annotated visuals, the system aids users with visual or reading impairments.
- **Analysts/Researchers:** Professionals can use it for post-hoc explanation of image-based data (medical images, engineering diagrams, etc.), providing both intuitive visuals and verbal reasoning.

Use-case examples include: explaining a physics diagram by highlighting forces and describing them step-by-step; walking through a complex game position (chess, Go, strategy game) with arrows and commentary; interpreting an annotated chart or map; or answering user questions about a UI screenshot with highlighted interface elements. In all cases, the system functions like an AI tutor or coach, giving both **visual cues** (arrows, boxes, labels on the image) and **spoken/textual explanations**.

Core Features

- **Image/Diagram Input:** Users can upload or draw a static image/diagram (PNG, JPG, PDF) through the web app.
- **Text/Voice Query Input:** The user can either type a question or speak it (using built-in speech-to-text). The system transcribes voice to text.
- **Vision Analysis:** The system uses computer vision (object detection, OCR, graph analysis) to parse the image's content. It identifies relevant components (text labels, shapes, objects, nodes, etc.) and their relationships. It may use models like CLIP, specialized diagram parsers (e.g. graph/flowchart detectors) or LLM-based vision (GLM, Gemini, etc.) to understand the scene.
- **Reasoning Engine (LLM):** A large language model (e.g. GPT-4 or GPT-4o) combines the parsed image content and the user's question to generate a coherent explanation. This involves causal, spatial or logical reasoning about the visual elements. Unlike simple captioning, the model produces an *explanatory answer*, not just an "answer" (e.g. it gives the *why* or *how*).
- **Multimodal Output:** The explanation is delivered in multiple modalities:
- **Textual Explanation:** A clear, multi-sentence answer describing the logic or process (similar to how a teacher would explain).
- **Annotated Visuals:** The original image is augmented with highlights (bounding boxes, arrows, color overlays, labels) indicating the key elements mentioned in the explanation. For instance, if explaining a circuit, wires are highlighted when discussed, with labels like "voltage source" or "current flow". State-of-the-art models (e.g. R2C networks) support producing such grounded visual rationales ².
- **Speech Output (TTS):** The text explanation is spoken aloud via a high-quality text-to-speech engine (to support hands-free or multi-modal learning).
- **Interactive Visualization (future enhancement):** Initially static annotated images are displayed, but the design will allow interactive elements: users can click on annotations to get more detail, or the system can animate steps. In future versions, short animated diagrams or video clips could be supported (though static images are the starting point).
- **Voice Response:** In addition to TTS reading the explanation, the system can engage in a voice-based dialogue (answering follow-up questions verbally). Using recent models, the system can handle back-and-forth spoken queries.
- **Domain Breadth:** The system is not limited to one field. It relies on large, general models and potentially domain-specific knowledge bases for specialized tasks (e.g. mathematical reasoning, chemistry rules). Initially it covers broad, common diagrams (educational charts, game boards, flow diagrams) and can be expanded.
- **Personalization & Feedback:** Users can rate or refine explanations ("That didn't answer my question" or "Highlight a different part"), and the system can adapt or hand off to follow-up queries. Logs of interactions can be used for improvement (research evaluation).

These features combine to create a **human-like explanation experience**: the AI looks at the image, interprets the user's intent, and simultaneously points to relevant parts of the diagram while narrating the reasoning, much like an expert tutor would. For example, prior work on vision-language explanation shows systems highlighting evidence and justifying answers in natural language ¹ ². Our system generalizes this to arbitrary domains, using the latest multimodal LLMs.

Functional Requirements

1. Image/Diagram Handling:

2. FR1.1: Accept image uploads (PNG/JPG/PDF) or allow user to paste a diagram.
3. FR1.2: Perform image pre-processing (e.g. standardizing size, enhancing readability).
4. FR1.3: Detect diagram elements (shapes, text, connections). Support static images initially; video support is optional for later.

5. Query Input & Processing:

6. FR2.1: Provide a text input field and a “press to speak” button.
7. FR2.2: Convert speech to text using a high-accuracy ASR engine.
8. FR2.3: Understand user intent from the query (classify question type if needed, e.g. “why”, “how”, “what”).

9. Multimodal Analysis:

10. FR3.1: Use Vision-Language Model(s) to convert image content to a structured form. This may include OCR for text, object detection/classification, spatial relations, graph parsing (for charts/flowcharts) and scene understanding. For example, models like GLM-4.5V can ground language queries to image regions ³.
11. FR3.2: Combine parsed image data and query text into a unified representation.

12. Reasoning & Explanation Generation:

13. FR4.1: Invoke a large language model (e.g. GPT-4/GPT-4o or open-source VLM) with the combined vision-text input. The prompt should encourage step-by-step explanation, not just answer.
14. FR4.2: Generate a coherent multi-sentence explanation of the relevant concept/process. The explanation must align with the visual evidence (e.g. “We highlight this part because...”).
15. FR4.3: Optionally, provide a brief summary (one-liner) and then a detailed explanation.
16. FR4.4: Ensure the explanation is suitable for a general audience (use simple language if targeted at students, per user’s choice).

17. Annotated Visual Output:

18. FR5.1: Determine which parts of the image correspond to concepts in the explanation. Use techniques like attention maps or object labels to map text to image regions ⁴.
19. FR5.2: Overlay visual annotations on the image: bounding boxes, arrows, highlights, and textual labels (captions). For example, label an arrow in the image as “electric current direction”.
20. FR5.3: Colors and annotations should follow UX guidelines (clear contrast, not overwhelming, optionally user-adjustable).

21. Voice Response:

22. FR6.1: Convert the final explanation text into natural-sounding speech (e.g. using Amazon Polly, Google WaveNet, or an open TTS system).

23. FR6.2: Provide audio playback controls (pause, replay).

24. User Interface:

25. FR7.1: Develop a web-based UI (accessible via browser).

26. FR7.2: Show side-by-side: (a) the annotated diagram, and (b) the explanation text (with timestamps or highlighting as audio plays).

27. FR7.3: Allow the user to click on the text or image to emphasize corresponding parts (bidirectional focus).

28. FR7.4: Provide controls for uploading images, recording speech, playing audio, and adjusting annotation visibility.

29. Interactivity & Follow-ups:

30. FR8.1: After the explanation, allow the user to ask follow-up questions (text/voice) about the same image. The system reuses the image context.

31. FR8.2: Permit simple commands like “show me where that is” or “explain the previous step again”.

32. FR8.3: Log user feedback (thumbs-up/down on explanation quality) for continual improvement.

33. Performance & Reliability:

34. FR9.1: Aim for response latency under 5 seconds for basic queries (web UI must feel responsive).

35. FR9.2: Handle typical diagrams of reasonable size (<2MB images, moderate complexity).

36. FR9.3: Ensure high accuracy of speech-to-text and image parsing (leverage robust pretrained models).

37. Security & Privacy:

38. FR10.1: Secure user-uploaded data: images and voice should not be publicly exposed.

39. FR10.2: Comply with privacy norms (e.g. user consent if storing data, GDPR rules if relevant).

Non-Functional Requirements

- **Scalability:** The system should support multiple simultaneous users (via scalable back-end, e.g. cloud deployment). Service APIs (LLM, vision) may be rate-limited, so design caching or batching if needed.
- **Availability:** Target high uptime (cloud hosting, failover). Provide error messages if a component fails (e.g. fallback if voice recognition fails).
- **Usability:** Intuitive UI. Provide example queries. Offer multi-language support in future (initially English, but architecture should allow other languages easily). Use best practices in HCI to make the experience smooth (clear buttons, loading indicators during processing, etc.).
- **Accessibility:** Follow accessibility guidelines (WCAG): keyboard navigation, screen reader labels, high-contrast modes. The multimodal nature helps (audio output assists visually impaired, text assists hearing impaired).

- **Maintainability:** Modular code design. Use standard frameworks (e.g. React for front-end, Python/Node for back-end). Well-documented APIs.
- **Extensibility:** Architect the pipeline so new models or features can be added (e.g. swap in a better vision model, add a translation feature, or support 3D diagrams later). For example, using a framework like LangChain allows easy integration of different LLMs and vision modules.
- **Performance:** Use optimized models (GPU/accelerator inference) to meet latency targets. Possibly use GPT-4o which processes vision + audio in one model ⁵, or at least GPT-4V for images and a state-of-art ASR (like Whisper) for voice.

System Architecture

The system follows a **pipeline architecture**. At a high level:

1. **Web Front-End (UI):** Browser interface (e.g. React app) for users. Allows image upload/display, text input, voice recording, and shows the annotated image plus text and audio controls.
2. **API/Back-End:** A server (e.g. Python Flask or Node) receives user inputs. It orchestrates calls to AI services and processes responses. It handles sessions, user state (current diagram), and aggregates outputs.
3. **Speech Module:**
 4. *Speech-to-Text:* On receiving voice, it calls an ASR engine (e.g. OpenAI Whisper, Google Speech API, or Azure Speech) to get the text query.
 5. *Text-to-Speech:* After generating the explanation text, it calls a TTS engine (AWS Polly, Google Cloud TTS, or a neural TTS like VITS) to get an audio file.
6. **Vision Module:** Processes the uploaded image:
 7. Run image preprocessing (resize, contrast).
 8. Apply OCR (if the diagram has text labels) and object detection/segmentation models to identify components. Open-source/industrial APIs (Google Vision, AWS Rekognition, or models like Detectron2, YOLO, Donut for diagrams) can be used.
 9. Optionally, use a vision-language model (e.g. GPT-4V, GLM-4.5V, Gemini, or Stable Diffusion-based analyzer) to produce a high-level caption or list of elements. For instance, GLM-4.5V can *ground* language to image: given a question, it yields bounding boxes for relevant objects ³.
10. Output: a structured description of the image (objects, spatial relations, text, any preliminary inference).
11. **Language & Reasoning Module:**
 12. Combine the query text (from user) and the image description into a single prompt or structured input.
 13. Call a large language model (OpenAI GPT-4o if available, or GPT-4V, or an open model like Gemma 3 or Anthropic Claude) to generate an explanation.
 14. The prompt should instruct the model to answer the user's question *and* explain the steps, referring to image elements (e.g. "Answer in steps, refer to the diagram, highlight evidence"). Modern multimodal LLMs (like GPT-4o) can handle text+image directly ⁵, or we simulate this by injecting image labels into the text.
 15. The LLM output is a textual explanation. If the model supports multi-turn, it can also plan annotations (e.g. output "Step 1: [text], highlight [object name]").
16. **Annotation Generation:**
 17. Parse the LLM's answer to find references to image parts. For example, if the text says "the red circle is the power source," we identify "red circle" in the image.

18. Use the vision model's earlier output (object labels/positions) to find coordinates. If available, use model-attention maps (Grad-CAM) to get heatmaps.
19. Generate overlay graphics: bounding boxes, colored highlights, arrows, and text labels. The content of labels comes from the explanation text.
20. Assemble the annotated image (PNG) to return to the user.
21. **Integration & Output:**
22. Send the annotated image and explanation text/audio back to the front-end. The UI presents them together.
23. Manage follow-up queries: The conversation context (original image, previous answer) is stored so the user can ask "Why?" or "What next?" and the system continues reasoning.

Figure: Example annotated diagram. In this illustrative food-web diagram, the system highlights key elements (yellow boxes, arrows) and would generate a verbal explanation linking these highlights to the user's query (e.g. explaining energy flow in the ecosystem). Our system similarly overlays highlights and labels on user-provided images to ground the explanation.

This modular architecture allows swapping or upgrading components. For instance, as new models arrive (GPT-5, more advanced vision models), they can replace older ones. Using a unifying framework (like LangChain or custom microservices) ensures each component (speech, vision, LLM) communicates via APIs. The web front-end can poll or receive push updates as the pipeline completes each stage.

Technology Recommendations

Given the user's requirement for **"latest"** and **"production-grade"** technology, we suggest the following stack (other choices are possible, but these represent strong options):

- **Large Language Model:** *OpenAI GPT-4o* (or GPT-4 Turbo with Vision). GPT-4o natively handles text, images, and audio in one model ⁵. It can accept the user's spoken question (converted to text) and image as input, and return a text answer. Its multi-modal capabilities simplify the pipeline. If proprietary APIs are not an option, open alternatives include Google's Gemini, Baidu's Ernie, or Meta's Llama 3/VL, or open-source VLMs like GLM-4.5V or Qwen3-VL. For example, GLM-4.5V offers strong visual grounding and step-by-step reasoning ³.
- **Vision Models:** Use pretrained vision models for general diagrams and objects. Options include:
- **Vision Transformers/Detectors:** Models like YOLOv8, DETR, or Mask R-CNN (available via HuggingFace) to detect shapes/objects.
- **OCR:** Tesseract OCR or Google Vision API for reading text in diagrams.
- **Diagram Understanding:** Specialized research models (e.g. Donut, Pix2Struct) for flowcharts or charts.
- **Vision-Language Models:** If using GPT-4o, it includes vision understanding. Otherwise, models like CLIP or ViT for feature extraction can assist the LLM.
- **Speech:**
- **ASR (Speech-to-Text):** OpenAI Whisper (open-source, high-quality, supports many accents), or cloud ASR (Google Speech-to-Text, Azure).
- **TTS (Text-to-Speech):** Neural TTS like Google WaveNet, Amazon Polly (Neural voices), or open TTS (Mozilla's TTS). Should support multiple languages/voices.
- **Front-End:** Modern web stack: React or Vue for UI, with HTML5/Canvas for displaying annotated images. Use WebSockets or REST for backend calls.

- **Backend:** Python (Flask/FastAPI) or Node.js. Python is convenient for AI libs. Use asynchronous calls to handle parallel API requests (speech, vision, LLM).
- **APIs & Frameworks:**
 - LangChain / LlamaIndex can orchestrate LLM prompts and integrations (image loaders, conversation memory).
 - Hugging Face Transformers / Diffusers for any local model inference.
 - Cloud services (AWS, Azure, or GCP) for scalable hosting and GPUs.
- **Data Storage:** A database (SQL/NoSQL) to log sessions, user feedback, and possibly fine-tune models on collected data.
- **Security:** HTTPS endpoints, OAuth for user accounts (if needed), secrets management for API keys.

These technologies together allow a powerful system. For example, GPT-4o's official announcement notes it **"accepts as input any combination of text, audio, image, and video and generates any combination of text, audio, and image outputs."** ⁵ This matches our needs exactly: we could even feed the image and the user's spoken query directly to GPT-4o and get an answer plus image highlights (if the API supports it). Similarly, open-source VLMs like GLM-4.5V can handle grounding: given a query, it "can locate and highlight specific visual elements in response to natural-language queries" ³.

Development & Implementation Plan

To build a **production-grade** system, we recommend an iterative development plan with clear milestones:

1. **Prototype Phase:**
2. Implement a basic pipeline with text input (no voice) and static images. Use off-the-shelf APIs: e.g. user uploads an image, types a question, backend calls an LLM (GPT-4V) to get an answer. Overlay highlights using model outputs or simplistic rules. This proves feasibility.
3. Example prototype: focus on one domain (like simple diagrams) to refine flow.
4. **Add Voice I/O:**
5. Integrate ASR for query input (start with a reliable open model like Whisper).
6. Integrate TTS for output (Google or open tools).
7. Ensure speech latency is low (streaming ASR).
8. **Improve Vision Understanding:**
9. Plug in specialized vision modules: test OCR on diagrams, object detection.
10. If available, incorporate a multimodal API (e.g. OpenAI's Vision API, or Azure's cognitive services) to directly answer "what's in this picture." This helps populate LLM prompts.
11. **Annotation Generation:**
12. Develop algorithms to parse LLM output and place annotations. Possibly use LLM itself: e.g. prompt GPT-4 to output JSON with text and coordinates.

13. Alternatively, use attention maps: tools like Grad-CAM on a vision model's features to get visual heatmaps, then draw boxes on top of those areas.

14. Create a UI component to overlay these transparently on the image.

15. User Interface & Experience:

16. Build the web app around the back-end pipeline. Focus on a clean layout: image on left, explanation text on right, audio controls above.

17. Add interactive features: clicking on explanation highlights the part on image (e.g. scroll to annotation), and vice versa.

18. Gather user feedback on UI ease-of-use and adjust (HCI iteration).

19. Scalability and Robustness:

20. Containerize services (Docker) and deploy on cloud (AWS/GCP). Use GPU instances for heavy inference.

21. Implement caching: if the same image+question appears, reuse past result.

22. Load-test to ensure the system can handle expected concurrency.

23. Domain Expansion & Fine-Tuning:

24. Gradually broaden beyond initial domain. If needed, fine-tune the LLM on explanation-style data (e.g. science QA datasets, diagram QA).

25. Incorporate domain-specific knowledge (e.g. an internal knowledge base of formulas for engineering diagrams).

26. Possibly allow users to specify context (e.g. subject area) to guide the model.

27. Evaluation and Research:

28. Collect metrics: accuracy of answers (via test questions), user satisfaction ratings, latency.

29. Conduct user studies: measure how well explanations improve understanding. This feeds into HCI research and validates the tool.

30. Iterate based on findings (e.g. rephrase explanations to be clearer, tweak annotation style).

31. Security & Privacy Review:

32. Ensure any user data (images, voice) are not stored without consent.

33. If the system is public-facing, implement user authentication and data encryption.

34. Launch & Monitoring:

35. Release an initial public version for a small group (students, educators).

36. Monitor usage, errors, and collect crash reports.
37. Plan for updates (new features, bug fixes) and research papers documenting novel aspects.

Throughout, maintain thorough documentation (for the API, for internal developers) and version control. Use CI/CD pipelines for automated testing of the functionality (e.g. regression tests with sample images/questions).

Non-Functional Considerations

- **Reliability:** Aim for 99% uptime. Use redundant servers and services.
- **Performance:** Model inference can be slow; use batching or streaming where possible. For example, stream ASR results to reduce wait time.
- **Ethics:** Since it's an explanation system, ensure the model's reasoning is not misleading. Warnings should be provided ("I am an AI tutor; verify complex answers"). Provide citations or confidence when uncertain.
- **Regulatory:** If used in classrooms, ensure compliance with student data privacy laws (e.g. COPPA, GDPR).
- **Cost:** Cloud inference (especially GPT-4/4o) can be expensive. Explore hybrid approach: use open models for most tasks, reserve GPT-4 for the hardest reasoning steps.

Future Directions

Once the core system is live, several enhancements are possible:

- **Dynamic Visuals:** Extend from static images to short videos or interactive simulations. The model could pause a video at key frames and explain them. (OpenAI notes GPT-4o can process video frames ⁵, albeit with constraints.)
- **Multilingual Support:** Expand to other languages (both UI and LLM) for international users.
- **Teacher Dashboard:** Enable educators to upload curriculum content (slides, diagrams) and get auto-generated lesson explanations or quizzes.
- **Customization:** Allow users to choose explanation style (concise vs. detailed, or analogies vs. formal).
- **Community & Research:** Release an API or SDK for researchers. Publish datasets (anonymized) of question-image pairs and explanations to further MXAI research.
- **Mobile App:** Complement the web app with mobile versions for on-the-go Q&A (taking photos of real-world diagrams).

Conclusion

In summary, the proposed system unifies cutting-edge **vision-language models** and **multimodal interfaces** to create a tutor-like explanation assistant. It draws on recent advances in explainable VQA ¹ and multimodal LLMs ⁵ ³. By meeting the listed requirements and leveraging appropriate technology, we can deliver a robust, user-friendly web application that empowers students and users in *every domain* to learn from diagrams in a natural, interactive way. The system's research and production facets ensure it remains both academically cutting-edge and practically useful, with continual improvement through user feedback and model updates.

Sources: Multimodal XAI literature and system design examples ¹ ⁴ ² ⁵ ³ and state-of-the-art model announcements (OpenAI GPT-4o) have informed this plan.

¹ A First Look: Towards Explainable TextVQA Models via Visual and Textual Explanations

<https://aclanthology.org/2021.maiworkshop-1.4.pdf>

² ⁴ Multimodal Explainable Artificial Intelligence: A Comprehensive Review of Methodological Advances and Future Research Directions

<https://arxiv.org/html/2306.05731v2>

³ Multimodal AI: A Guide to Open-Source Vision Language Models

<https://www.bentoml.com/blog/multimodal-ai-a-guide-to-open-source-vision-language-models>

⁵ Hello GPT-4o | OpenAI

<https://openai.com/index/hello-gpt-4o/>